Literate Programming in the Era of LLMs

Ramesh Subramonian

March 1, 2026

Abstract

LLMs have taken the programming world by storm. How does this change the role of the software engineer? Assuming that the cost of writing code will approach zero, what does a software engineer do? We posit that there will be some fundamental changes in the way we will build software. (1) We will focus more on writing unambiguous, semi-formal specifications than we have traditionally done (2) We will deploy more custom code, tailored to the specific need of the application, thereby reducing code bloat (3) We will adopt just-in-time manufacturing approach to software deployment In this paper, we use an example to demonstrate how these principles are put into practice.

1 Introduction

Traditionally, the cost of writing software (and modifying it when new needs arose) was so high that the design of code was optimized for maintainability. Most seasoned software engineers will have experienced the sinking feeling when asked to add new requirements to a large code base, the fear of not fully understanding whether a fix here would cause a blow out there. We would argue that assuming zero cost of (bug-free) software development flips this paradigm on its head. This moves the burden from refactoring code to making sure that the specifications are unambiguous and consistent and meet the business requirement precisely.

Good software engineering was never about churning out lines of code. To quote Lattner, *Writing code has never been the goal. Building meaningful software is.* [Lat26]. Instead, it was about designing systems that met the requirements. Djkstra's claim was never more relevant than it is today — "*Computer science is no more about computers than astronomy is about telescopes.*" Writing efficient code was a means to an end. There have been a series of improvements in that process, LLMs representing the latest and admittedly most unexpected quantum jump in efficiency. Writing formal specifications is a declaration of intent. Until the singularity arrives, specifications represent the safest bet that we will get what we want when we no longer control the translation from intent to a higher level programming language, en route to a further translation to machine instructions.

Which brings us to the central question that this paper seeks to answer.

How much code should a coder code if Claude Code could code code?

1.1 Organization

In the rest of the paper, we will flesh out the details of the three-step approach suggested below.

1. control the data, the format and inter-relationships
2. write specifications for new operators and have the LLM write code in a high level programming language which you can review
3. deploy functionality as needed, at the cost of a small delay when you ask for something new

In Section 2, we present the example used to ground the claims made above. In Section 3, we describe how our approach to software development changed fundamentally with the inclusion of LLMs. In Section 4, we make the approach and philosophy concrete by delving into the details of the implementation.

2 Motivation

Some years ago, we were developing a system similar to Spark in terms of distributing many independent pieces of computation over a network. We needed a data format that could be transmitted and operated upon efficiently. Examples of viable data formats were BSON [Mon] and AVRO [AVR]. Ultimately, we decided it best to roll our own format based on the following requirements and usage patterns

1. ease of adding fixed-width custom types
2. optimizing computations on the data without moving or transforming it
3. allowing for in-situ operations, although most operations are write-once.
4. optimizing for the case where data is read more often than it is written.
5. reluctance to depend on software which was unashamed to admit that "*The documentation is rather sparse right now but we'll be adding more information soon.*"[1]

The development work could be broadly divided into two parts

1. Definition of the data format and writing helper functions which allowed the creation and modification of data packets in `QDF` format and access to elements of such a packet
2. Definition and development of operators that read from (and occasionally wrote to) the data packet. A simple example of an operator is a dot product which accesses two numeric arrays of equal length and returns a number.

Over a period of four years, we found that the changes in the underlying data format were both infrequent and minor. As a consequence, the work on getters and setters was minimal. Instead, most of the development work involved

[1] `https://avro.apache.org/docs/1.12.0/api/c/`

1. writing new operators.
2. optimizing the performance of existing operators

This led to the following questions

1. Could the use of LLMs have reduced the development and maintenance cost?
2. If so, what would the software architecture of such a system look like?

Giving GenAI the benefit of the doubt, we assumed the answer to the first question would be Yes and proceeded to answer the second question.

3 Designing for LLMs

3.1 Choice of Language

We chose to build our system using C and Lua and LaTeX. There is nothing sacrosanct about these choices. Other choices of compiled language and interpreted language and document formatting language are likely to work just as well. The reasons for this choice were

1. The Ousterhout dichotomy. This is a principle of software design that I have found very useful. It argues for the judicious blending of a low-level, statically-typed systems language like C and a high-level, dynamic scripting language like Lua [Ous].
2. Completeness. It was very much our intent to have a fully functional system of moderate complexity which would allow us to suss out any gotchas that a "Hello World" program might have glided by. The author is sufficiently conversant in these languages to meet this goal.
3. Ease of exposition. The simple syntax of C and Lua make the code snippets widely readable
4. Ease of implementation. Lua was designed to inter-operate with C [Ier84]
5. LaTeX, when combined with `make`, allows us to "compile" documentation from disparate sources. It allows us to create different targets from common sources.

3.2 Control the Data

We took our inspiration from the quote attributed to Linus Torvalds — *"Bad programmers worry about the code. Good programmers worry about data structures and their relationships.* Hence, we focused our efforts on designing an efficient data format (Section 3.2.1) and and defining how it would be accessed (Section 3.2.2). This information is passed to the LLM in the form of a LaTeX file. For human readability, some scripted transformations are performed. For example, a Lua table of the form

```
return { Q0 = "void", I32 = "int32_t", FP32 = "float", FP64 = "double", }
```

is formatted as Table 1 using a Lua script invoked by `make`.

3.2.1 Define the Data Format

We wanted the data format to resemble JSON but to have efficient representations of commonly used types e.g., dataframes. Consider the JSON type `number`. This is stored in QDF as a 16 byte packet where

1. Byte 0 contains JSON type information — `number` in this example.
2. Byte 1 contains data type information e.g., `int32_t` or `double`
3. Byte 2 is the successor offset. While this is usually 0, it helps tell us where the successor QDF, if any, is located. This is used when we want to make sure that certain data structures are memory aligned, helpful for vector instructions.
4. Byte 3 is unused.
5. Bytes 4 to 7 contain an offset to the parent packet, if any. For example, in the JSON array `[ true, "abc", 123 ]`, the number 123 needs to know the start of the parent array of which it is a part. Similarly, the successor offset in Byte 3 tells us how much padding exists between the boolean `true` and the string `"abc"`.
6. Bytes 8 to 15 contain the actual value

3.2.2 Define the Accessors

After defining the data format, we implemented a number of helper functions which broadly fell into these categories

1. readers or getters e.g., calling `get_jtype()` on a QDF packet tells us what JSON type it is i.e., `nil, number, string, ...`
2. writers or setters
3. makers or constructors e.g., calling `make_number()` with the type and value of the number to be created as arguments,
4. destructors were largely handled by Lua's garbage collection
5. checkers, which validated the syntactic structure of a data packet. These can be thought of as invariants, inspired by [Hoa69].

Q Type	**C Type**
Q0	`void`
I32	`int32_t`
FP32	`float`
FP64	`float`

Table 1: Qtypes to C Types

3.2.3 Test the Accessors

Once the accessors were in place, it was necessary to write unit tests which would give us confidence in their implementation. Currently, tests are hand-written although this is a perfect candidate for handing off to the LLM.

3.2.4 Define the Operators

We took our inspiration from Knuth: "*Let us change our traditional attitude to the construction of programs: Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer to do.*" [Knu84].

Taking this last sentence to its logical conclusion, we chose to concentrate **entirely** on the human explanation, leaving the job of instructing the computer what to do to the LLM. In our case, we chose to use LaTeX for specifications and have the LLM write the code in Lua, C and ISPC [PM12] The author's familiarity with these languages allowed for code review. If LLMs live up to the trajectory claimed for them, we expect to gain confidence in them and reduce the need for oversight.

4 Making it concrete

Let us assume that having defined the data format, we wish to add an operator `coalesce` [2]. We walk the reader through the process, step by step, explaining what the developer does and what the system does in response

1. Text in red font is what the developer does
2. Text in blue font is what the LLM does
3. Apart from that, most of this section con concentrates on what the system does

4.1 Starting Out

```
lQDF = require 'lQDF'
```

This informs the system about the data definitions and data accessors. To those familiar with LuaJIT [Pal], we have

1. `ffi.cdef`'d the function declarations
2. `ffi.load`'d the library that implements the accessors

4.2 Registering an operator

```
local X = require 'coalesce'
lQDF.Q.register('coalesce', X)
```

[2] https://www.rdocumentation.org/packages/dplyr/versions/1.0.10/topics/coalesce

In the first line, we define `X`, the specification of the operator, Section 4.2.1. It consists of the following

1. `generic_tex_spec_file`: LaTeX file containing type-agnostic specifications. See Section D (captioned **Coalesce**) of Appendix for an example
2. `get_subs`: a function which produces substitutions needed to specialize the above LaTeX file based on the types of the arguments
3. `lua_calling_spec`: table containing specification used to create a function `run()` which is used at runtime to create (if necesssary) and execute a C function

In the second line, the call to `Q.register` causes the following

1. Records the information provided
2. Dynamically generates the code for a function `run()` using `lua_calling_spec`. Currently, this is done by a human authored script. Future work could see this be passed to the LLM as well.
3. Dynamically compiles the code generated above using Lua's `loadstring()` into a variable of type `function`
4. Adds `coalesce` as a key to the table `lQDF`, the value of which is the function `run`

4.2.1 Coalesce specification

```
-- Creating a new operator involves writing a file like this
local C = {} -- table of stuff to return
C.test = function() print("test function for coalesce") end
C.get_subs = function (x, y)
  local operator = "coalesce"
  local cfunc = { operator , "_", x:qtype(), "_", y:qtype(), }
  cfunc = table.concat(cfunc, "")
  return {
    operator = operator, -- NECESSARY
    -- following for substitutions needed by LaTeX spec file
    __QTYPEX__ = x:qtype(),
    __QTYPEY__ = y:qtype(),
    __CFUNC__ = cfunc,
  }
end
C.generic_tex_spec_file = "coalesce.tex"
-- convention that in_args passed before out_args
C.lua_calling_spec = {
  operator = "coalesce",
  in_args =  {
    { name = "x", qtype = "lQDF", rw = "read_only", },
    { name = "y", qtype = "lQDF", rw = "read_only", },
  },
```

```
  out_args  = {
    { name = "z", qtype = "lQDF", }
  },
}
return C
```

4.3 Calling an operator

```
local x = lQDF( 1, 2, 3,
local y = lQDF( 4, 5, 6, )
local z = lQDF.coalesce(x, y)
```

The first two lines create `x, y`, which are objects of type lQDF. The third line creates `z`, also an object of type `lQDF`. Let's take a peek under the covers and see what the system does to create `z`.

1. Based on the types of the input arguments, `x, y`, determine whether a specialized function for the job exists. For now, let's assume it doesn't.
2. Make substitutions in the generic LaTeX file e.g., this is where we would find that the `qtype` of `x,y` is `FP64`, as a result of which we would need to create a function whose name is `coalesece_FP64_FP64`
3. Assemble a larger LaTeX file which contains context which does not change across operators. This is the information in Sections 3.2.1 and 3.2.2.
4. Use `pdflatex` to create a PDF with detailed instructions for the LLM. See the Appendix for an example of what is sent to the LLM. Note that the developer writes only the portion of this document specific to their operator
5. POST the PDF to the LLM. We also post relevant `.h` (created by the system architect not the operator writer) that define `structs, enums, macros`, such as
   (a) `qdf_struct.h`
   (b) `qtype.h`
   (c) `jtype.h`
   (d) `macros.h`
   (e) function declarations for all helper functions

   We instruct the LLM to return
   (a) the function declaration `.h` file
   (b) the function definition `.c` file
   (c) commands needed to create a `.o` file and a `.so` file. The LLM gives us information such as
      i.
      ii. compiler to be used e..g, `gcc` or `ispc`
      iii. include paths
      iv. libraries that may need to be linked

v. compiler/linker flags

6. Execute above commands to create a `.so` file.
7. `ffi.cdef` the `.h` file and `ffi.load` the `.so` file created above
8. The previous steps are skipped over if they had been executed before. The system remembers past work. Coming across a new symbol which necessitates the above work can be thought of as a cache miss. This is an acceptable tradeoff **if**
   (a) these cache misses are infrequent
   (b) after an initial warm-up phase, the system settles into a steady state
   (c) the deployment is not sensitive to the latency they introduce,
9. Provide C access to the Lua objects `x, y, z`
10. Execute the C function using the pointers provided above as arguments
11. Return the Lua objects specified in `out_args` in Section 4.2.1.

4.4 Saving and Restoring

What happens when a session ends? If we wish to save operators created in this session for use in subsequent sessions, we execute

```
lQDF.Q.save()
```

This records all necessary artifacts on the file system e.g., LaTeX files, `.pdf` files, ...

When we start a new session, we do not need to start from a blank slate. Instead, we can pick up from where we left off as show below.

```
lQDF.Q.restore()
local z = lQDF.coalesce(lQDF(1,2,3,), lQDF(4,5,6,))
```

5 Discussion

I started this journey as a curious sceptic. I reached this milestone as a cautious optimist. I was amazed at the quality of the code produced. But I didn't leave me with a sense of dread that something I took pride in could be replicated so easily. Instead, it felt liberating. I had found the L3 engineer that I had always wanted to work with. Maybe tomorrow I will find the colleague I always wanted to pair program with.

6 Conclusion

The rapid pace of progress in AI makes prognostication difficult. Does one ignore it and wait for the bubble to burst? Does one embrace it entirely like a product manager of mine who wondered why it took me so long to convert his PowerPoint wireframes into working code? This paper represents a measured adoption of the seemingly miraculous powers of AI.

We agree with Lattner's assessment that *As writing code is becoming easier, designing software becomes more important than ever. As custom software becomes cheaper to create, the real challenge becomes choosing the right problems and managing the resulting complexity.* [Lat26].

6.1 Speculation

1. Has AI, for the software engineering domain, become a commodity? For the simple example cited in this paper, I could not find any appreciable difference in code quality across several LLMs — ChatGPT, DeepSeek, Gemini, ...

   One could argue that that was because the context provided was concise and precise. But isn't that what we would want anyway? Rather than treating code as all important and documentation and comments as annoyances, it seems right to focus effort on communicating intent.

   So, while more recent models are defying belief by solving Math Olympiads, does one need that power in the next few years to produce high quality software economically? I suspect not.

2. Benioff's theatrics in 2000 declaring that "software is dead" may well have come home to roost. Why would I pay for software when what I am really paying for is specifications? I believe the time has come for disciplined software engineering that uses LLMs for code generation to disrupt the enterprise software market. Customizing a system for different businesses will not require the same effort that it has historically required.

3. Many software consulting houses (here's looking at you Infosys, TCS, ...) will suffer but not because the need for software will disappear. Instead, they have hired, trained and promoted for vocational skills. I suspect that they are unprepared to pivot to a realm where engineering skills dominate. Similarly, many universities in India are really trade schools masquerading as bastions of higher learning. I do **not** use the term "trade school" disparagingly. I think they have a valuable purpose. I do hope that the mechanic who tunes up my car has attended one of those. But that doesn't mean that I'd hire her to design my dream car.

4. Would there soon come a time where a software engineer apologizes for writing code instead of a specification with the quote various atttibuted to Cicero, Pascal, Mark Twain

   > *I didn't have time to write a short letter, so I wrote a long one instead.*

5. Would LLMs change the way we design programming languages? So far, the target of human effort has been code for a high-level programming language. How would a specification language differ from what we have traditionally thought of as a programming language?

6. Many years ago, I graduated in the middle of a painful recession. A friend of mine persuaded a hiring manager at Oracle to take a call from me. I picked up the handset from the phone in the lab and dialed out. A gruff voice acknowledged me.

I launched into my rehearsed pitch. "I've just graduated with a PhD in theoretical Computer Science. " He cut me off. "We want workers, not thinkers." and the line went dead. "*Workers, not thinkers*" — the phrase has haunted me these many years. Maybe the shoe is finally on the other foot?

6.2 Future Work

Validating the approach proposed will take more than one example. Among my targets are

1. Karpathy's one C file implementation of Llama 2 inferencing [3]. Starting from a crisp definition of the data structures — network and weights — and a set of math formulas, could the implementation come for free?
2. A web application testing the applicability of this technique to languages I am not comfortable with — HTML and Javascript
3. Suggestions are welcomed

6.3 Open Questions

We are sharing this work before all the rough edges have been smoothed out in the hope of getting feedback. We would especially like suggestions on

1. packaging, versioning, ...
2. extending code generation to other parts of the system, such as unit testing
3. cross-testing different LLMs against each other
4. viable replacements for the role C and Lua played in this implementation

References

[AVR] Avro. `https://avro.apache.org/docs/`.

[Hoa69] Charles Antony Richard Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.

[Ier84] Roberto Ierusalimschy. A Look at the Design of Lua. *Communications of the ACM*, 27(2):97–111, 1984.

[Knu84] Donald E. Knuth. Literate Programming. *The Computer Journal*, 27(2):97–111, 1984.

[Lat26] Chris Lattner. The Claude C Compiler: What It Reveals About the Future of Software. `https://www.modular.com/blog/the-claude-c-compiler-what-it-reveals-about-the-future-of-software`, February 2026.

[3] `https://github.com/karpathy/llama2.c`

[Mon] MongoDB. Bson. https://www.mongodb.com/resources/languages/bson#what-is-bson.

[Ous] The Ousterhout dichotomy. https://gist.github.com/nat-418/bdccb2428cde750c9f42bf9bbc1a50d3.

[Pal] Mike Pall. Luajit. https://luajit.org/luajit.html,.

[PM12] Matt Pharr and William R. Mark. ISPC: A SPMD Compiler for High-Performance CPU Programming. In *Proceedings of Innovative Parallel Computing (InPar) 2012*. IEEE, May 2012.

A QDF Data Layout

This section defines the layout of data in a QDF.

A.1 Notations

Notation 1 *The symbol $\perp$ will be used to indicate undefined*

A.2 Conventions

Convention 1 *The* `gcc` *compiler will be used to compile C code.*

Convention 2 *The* `ispc` *compiler will be used to compile ISPC code.*

Convention 3 *The* `gcc` *compiler will be used to link* `.o` *files and create a* `.so` *file.*

Convention 4 *All calls to function that return an integer status should be followed by a call to the macro* `cBYE(status);` *For example* `status = foo();` *should be followed by* `cBYE(status);`

Convention 5 *For all functions that return an integer (with value 0 on success and non-zero on error) should start with the line*

int status = 0;

and end with the line

return status;

A.3 qtype

The enum `qtype_t` is defined in file `qtype.h` and is reproduced below

A.4 q2c

The Lua table q2c shown in Table 2 is defined in `q2c.lua`. It is a mapping between a value of type `qtype_t` and a C type.

A.5 jtype

The enum `jtype_t` is defined in file `jtype.h` and is reproduced below

```
typedef enum {
  j_error = 0,
  j_undef,
  j_nil,
  j_bool,
  j_string,
  j_number,
  j_date,
  j_array,
  j_object,
  // j_hashtable
} jtype_t;
```

A.6 QDF struct

The struct `qdf_rec_type` is defined in file `qdf_rec_type.h` and is reproduced below

```
typedef struct _qdf_rec_type {
  void *data;
  uint32_t size; // must be a multiple of 8
  bool is_mmap; // true => we have mmapped data not malloc'd it
  bool is_foreign; // true => do not free() or munmap()
  bool is_read_only; // true => don't modify
} QDF_REC_TYPE;
```

B Helper functions

Helper functions are provided in `libqdf.so`. All function declarations are available in `qdf.h` There are three kinds of helpers

1. readers or getters — Section B.1
2. writers or setters — Section B.5
3. makers — Section B.2

Q Type	C Type
I32	`int32_t`
TM	`struct tm`
FP32	`float`
Q0	`void`

Table 2: Qtypes to C Types

B.1 Read only helper functions

Functions listed in Table 3 provide read only access to portions of a qdf. They take as an argument an immutable pointer to `qdf_rec_type` and do **not** modify the location pointed to.

Return	**Name**	**Args**
bool	`chk_qdf`	(const `qdf_rec_type` * const x)
`qtype_t`	`get_qtype`	(const `qdf_rec_type` * const x)
`jtype_t`	`get_jtype`	(const `qdf_rec_type` * const x)
uint32_t	`get_length`	(const `qdf_rec_type` * const x)
const char *	`get_read_arr`	(const `qdf_rec_type` * const x)
const bool *	`get_read_nulls`	(const `qdf_rec_type` * const x)
uint32_t	`get_num_keys`	(const `qdf_rec_type` * const x)

Table 3: List of read only helper functions

B.1.1 `chk_qdf`

Returns `true` if x is syntactically valid; `false`, otherwise

B.1.2 `get_qtype`

Returns `qtype_t` of x

B.1.3 `get_jtype`

Returns `jtype_t` of x

B.1.4 `get_length`

If `jtype_t` of x is `j_array`, then returns length of array; else, returns 0.

B.1.5 `get_read_arr`

- If `jtype_t` of x is `j_array`, then returns `NULL`.
- If `qtype_t` of x is Q0, then returns `NULL`.
- Else, returns a pointer to the 0th element of the array.

The data pointed to cannot be modified.

B.1.6 `get_read_nulls`

- If `jtype_t` of x is `j_array`, then returns `NULL`.
- If `qtype_t` of x is Qerr or Q0, then returns `NULL`.
- If x does not have null values, then return `NULL`.

Else, returns a pointer to the 0th element of the bool array, nnx, such that

$$nnx[i] \Rightarrow x[i] \neq \perp$$

$$\neg nnx[i] \Rightarrow x[i] = \perp$$

The data pointed to cannot be modified.

B.1.7 `get_num_keys`

If `jtype_t` x is `j_object`, then returns number of keys; else, returns 0.

B.2 Maker helper functions

Functions listed in Table 4 help you create or make a `qdf_rec_type` .

1. The first argument is always a pointer to `qdf_rec_type` , the elements of which will be set by this function
2. They may have other arguments as well.
3. The return value is an integer status which is 0 for success and non-zero for failure.

Name	**Arguments**
`make_uniform_array_with_nulls`	`qdf_rec_type` x
	`qtype_t` q
	`uint32_t` n
`make_uniform_array`	`qdf_rec_type` x
	`qtype_t` q
	`uint32_t` n

Table 4: List of maker helper functions

B.3 `make_uniform_array`

Creates an empty QDF meant to store a uniform array that does **not** have null values.

B.4 `make_uniform_array_with_nulls`

Creates an empty QDF meant to store a uniform array that **does** have null values.

B.5 Writable helper functions

Functions listed in Table 5 modify the elements of a variable of type `qdf_rec_type` .

1. The first argument is always a pointer to `qdf_rec_type` which can be used to modify the location pointed to.
2. They may have other arguments as well.
3. The return value is an integer status which is 0 for success and non-zero for failure.

Name	Argument 2	Argument 3
TODO	TODO	TODO
TODO	TODO	TODO

Table 5: List of read only helper functions

C Macros

The following macros should be used when possible

1. Print file and line number

   ```
   #define mcr_WHEREAMI{fprintf(stderr, "Line %3d, File %s\n", __LINE__, __FILE__
   ```

2. Print file and line number, set status to `x` and jump to label `BYE`

   ```
   #define mcr_go_BYE(x) { WHEREAMI; status = x ; goto BYE; }
   ```

3. If value of `x` is not 0, then print file and line number and jump to label `BYE`

   ```
   #define mcr_cBYE(x) { if ( (x) != 0 ) { go_BYE((x)) } }
   ```

4. Used to determine whether i^{th} element of x is null

   ```
   #define mcr_is_null(x, i) ( ( x != NULL ) && (x[i] == false) )
   ```

5. Used to determine whether variable `x` of type pointer to `qdf_rec_type` is empty

   ```
   #define is_empty(x) ( ( x->data == NULL ) && ( x->size == 0 ) )
   ```

6. Used to determine whether variable `x` of type pointer to `qdf_rec_type` is not empty

   ```
   #define is_not_empty(x) ( ( x->data != NULL ) && ( x->size != 0 ) )
   ```

D Coalesce

Create optimized C code for the function defined in this section.

- Let x be an immutable pointer to immutable `qdf_rec_type` .
- Let y be an immutable pointer to immutable `qdf_rec_type` .
- Let z be a pointer to `qdf_rec_type` .
- qx := F8// qtype(x)
- qy := F8// qtype(y)
- cqx := q2c[qx] C type corresponding to qx as defined in Lua table q2c
- cqy := q2c[qy] C type corresponding to qy as defined in Lua table q2c
- jx := `get_jtype`(X)
- jy := `get_jtype`(Y)
- nx := `get_length`(X)
- ny := `get_length`(Y)
- const cqx * const vx = `get_read_arr`(x)
- const cqy * const vy = `get_read_arr`(y)
- const bool * const nnx = `get_read_nulls`(x)
- const bool * const nny = `get_read_nulls`(y)

Function signature is

1. name is coalesce_F8_F8
2. Return value is of type integer. Return value is 0 on success, -1 on failure.
3. Arguments are x, y, z specified above

Function fails under the conditions listed below. Upon failure, execute macro `mcr_go_BYE(-1)` specified in Section C

1. Fails if x == NULL
2. Fails if `is_empty(x)`
3. Fails if y == NULL
4. Fails if `is_empty(y)`
5. Fails if x == y
6. Fails if z == NULL
7. Fails if `is_not_empty(z)`
8. Fails if qx $\neq$ qy
9. Fails if jx $\neq$ `j_array`
10. Fails if jy $\neq$ `j_array`

11. Fails if nx $\neq$ ny
12. Fails if nx == 0

Function logic is as follows.

1. Set $nz \leftarrow nx$
2. Set $qz \leftarrow qx$
3. Execute `make_uniform_array`(z, qz, nz)
4. cqx *vz := `get_write_arr`(z)
5. bool *nnz := `get_write_nulls`(z)

Execute following code **foreach** $i \in [0, nz - 1]$

```
if (  not mcr_is_null(nnx, i) ) then
  vz[i] := vx[i]
  nnz[i] := true
else
  if ( not mcr_is_null(nny, i) ) then
    vz[i] := vy[i]
    nnz[i] := true
  else
    vz[i] := 0
    nnz[i] := false
  endif
endif
```

E Instructions to LLM

Generate and return the following

1. optimized C code for the function coalesce_F8_F8 described above as
   (a) a `.c` file named coalesce_F8_F8.c containing the function definition
   (b) a `.h` file named coalesce_F8_F8.h containing the function declaration
2. the command to compile coalesce_F8_F8.c, link necessary libraries and create a `.so` file named libcoalesce_F8_F8.so